Note de Curs: Baze de Date și SGBD

Cuprins

1 Curs 1: Generalități despre Baze de Date și SGBD **3**

1.1 Concepte de bază: Baza de date, SGBD și Dicționarul Datelor 3

1.2 Personalul implicat și Arhitectura Sistemului 3

1.3 Niveluri de abstractizare și Independența Datelor 4

1.4 Limbajele Sistemelor de Baze de Date 4

1.5 SGBD (Sisteme de Gestiune a Bazelor de Date) 4

1.5.1 Obiectivele unui SGBD 4

1.5.2 Dezavantajele SGBD-urilor 5

1.5.3 Structura unui SGBD pe module 6

1.5.4 Arhitectura SGBD 6

1.6 Evoluția Bazelor de Date: De la Ierarhic la Suport Decizional 7

2 Cursurile 2, 3: Proiectarea Bazelor de Date (Modelarea E/R) **8**

2.1 Modelarea Entitate-Relație 8

2.1.1 Modelul de Date: Definiție și Componente 8

2.1.2 Modelarea semantică a datelor 8

2.2 Diagramele Entitate-Relație (E/R) 8

2.3 Cazuri Speciale în Modelarea E/R 9

2.4 Algoritmul de Proiectare E/R - Exemplu Practic 13

2.5 Modelul E/R Extins 13

2.6 Deficiențe ale modelului E/R 14

3 Curs 4 **15**

3.1 Modelul relațional 15

3.1.1 Constrângeri de integritate 16

3.2 Diagrama conceptuală 16

3.2.1 Transformarea entităților 16

3.2.2 Transformarea relațiilor 17

3.2.3 Transformarea atributelor 17

4 Curs 6: Exemplu de Diagramă Conceptuală 19

5 Curs 7: Algebra Relațională 21

5.1 Operatorul PROJECT 22

5.2 Operatorul SELECT 22

5.3 Operatorul UNION 22

5.4 Operatorul DIFFERENCE 23

5.5 Operatorul INTERSECT 23

5.6 Operatorul PRODUCT 24

5.7 Operatorul JOIN 24

5.7.1 NATURAL JOIN 25

5.7.2 θ-JOIN 25

5.7.3 SEMI-JOIN 26

5.7.4 OUTER JOIN 26

5.8 Operatorul DIVISION 26

6 Curs 8: Evaluarea și Optimizarea Interogărilor 28

6.1 Procesarea interogărilor 28

6.2 Ordinea de execuție a operațiilor 30

6.3 Proprietățile operatorilor algebrei relaționale. Optimizare 31

7 Secțiunea PL/SQL: Variabile și Structuri de Control 34

7.1 Tipuri de Date și Declararea Variabilelor 34

7.2 Structuri Decizionale 34

7.3 Structuri Repetitive (Bucle) 34

7.4 Cursori în PL/SQL 35

7.5 Gestiunea Excepțiilor 36

1 Curs 1: Generalități despre Baze de Date și SGBD

Conceptul de gestiune a datelor a evoluat semnificativ pentru a răspunde nevoilor de stocare, organizare și extragere eficientă a informațiilor.

1.1 Concepte de bază: Baza de date, SGBD și Dicționarul Datelor

Baza de Date (BD) este un ansamblu structurat de date coerente, organizate fără redundanță inutilă, astfel încât acestea să poată fi prelucrate eficient de mai mulți utilizatori într-un mod concurent. Este o colecție de date persistente folosite de sistemele de aplicații ale unei anumite „întreprinderi" (o organizație cu reguli proprii de funcționare și mulțimi de date referitoare la modul său de operare). Datele dintr-o BD sunt integrate și partajate.

Sistemul de Gestiune a Bazelor de Date (SGBD / DBMS) este un produs software complex care asigură interacțiunea cu baza de date; acesta permite definirea, consultarea și actualizarea datelor. Toate cererile de acces la date sunt tratate și controlate de acest sistem; printre cele mai cunoscute SGBD-uri se numără MySQL, ORACLE și Microsoft SQL Server.

Dicționarul Datelor (Metabaza de Date)

Dicționarul datelor este catalogul de sistem ce conține „date despre date". Este structurat și administrat ca o bază de date în sine și conține:

- Descrierea tuturor obiectelor bazei de date (tabele, indecși, vizualizări).
- Starea curentă a acestor obiecte.
- Diversele constrângeri de securitate și de integritate impuse sistemului.

Dicționarul poate fi interogat ca orice altă bază de date.

1.2 Personalul implicat și Arhitectura Sistemului

Un sistem de baze de date include baza de date propriu-zisă, SGBD-ul, dicționarul datelor, resursele hardware și reglementările administrative. Elementul uman este împărțit în patru mari categorii:

1. **Administratorul de date (DA - Data Administrator):** Un manager care decide ce date trebuie stocate și stabilește regulile de întreținere a acestora.
2. **Administratorul bazei de date (DBA - Data Base Administrator):** Profesionistul IT responsabil cu ansamblul activităților tehnice: creează baza reală, implementează controlul tehnic, asigură performanța, securitatea și recuperarea datelor.
3. **Proiectanții și programatorii:** Cei care acoperă proiectarea logică și fizică a bazei de date și scriu aplicațiile.
4. **Utilizatorii finali:** Cei care accesează și exploatează informația.

1.3 Niveluri de abstractizare și Independența Datelor

Pentru a ascunde complexitatea stocării fizice față de utilizator, arhitectura bazelor de date este organizată pe trei niveluri de abstractizare *(arhitectura ANSI/SPARC)*:

- **Nivelul fizic (intern):** Descrie *cum* sunt stocate fizic datele pe disc (biți, octeți, adrese, indecși).
- **Nivelul conceptual:** Reprezintă structura semantică globală a bazei de date, *ce* date sunt stocate și relațiile dintre ele, integrând toate cerințele utilizatorilor. Este viziunea programatorilor de sistem.
- **Nivelul extern (virtual / vizualizări):** Modul în care diferiți utilizatori percep datele. Fiecare utilizator vede doar submulțimea de date relevantă pentru el.

O consecință directă a acestei arhitecturi este **Independența Datelor**:

- **Independența fizică:** Permite modificarea schemei fizice (ex. schimbarea hard-diskului, adăugarea unui index) fără ca schema conceptuală, schema logică sau programele de aplicație să fie afectate.
- **Independența logică:** Permite modificarea schemei conceptuale (ex. adăugarea unui tabel nou sau a unui atribut) fără ca vizualizările utilizatorilor (schema externă) și aplicațiile existente să necesite rescriere.

1.4 Limbajele Sistemelor de Baze de Date

Interacțiunea cu SGBD-ul se face prin trei categorii de limbaje:

1. **LDD (Data Description Language):** Pentru definirea entităților, a atributelor, a relațiilor, și a constrângerilor de securitate și integritate.
2. **LMD (Data Manipulation Language):** Pentru adăugarea, modificarea, ștergerea și consultarea datelor. Poate fi procedural (specifică *cum* se obține rezultatul) sau neprocedural (specifică *ce* se dorește obținut, cum e standardul SQL).
3. **LCD (Data Control Language):** Pentru asigurarea confidențialității, salvarea/recuperarea în caz de defecțiuni și rezolvarea concurenței.

1.5 SGBD (Sisteme de Gestiune a Bazelor de Date)

1.5.1 Obiectivele unui SGBD

Un **SGBD** are următoarele **obiective fundamentale**:

- **Independența fizică:** independența structurilor de stocare în raport cu structurile de date din lumea reală

- **Independența logică:** fiecare grup de lucru poate să cunoască doar o parte a semanticii datelor, să vadă doar o submulțime a datelor și numai sub forma în care le dorește
- **Prelucrarea datelor de către neinformaticieni**
- **Administrarea centralizată a datelor**
- **Coerența datelor:** informația trebuie să satisfacă constrângeri statice sau dinamice, locale sau generale
- **Neredundanța datelor**
  - Administrarea coerentă a datelor trebuie să asigure neduplicarea fizică a datelor
  - **Pentru a realiza performanțe referitoare la timpul de acces la date și răspuns la socilitările utilizatorilor, se acceptă o anumită redundanță a datelor**
- **Partajabilitatea datelor**
  - Aplicațiile pot să partajeze datele din baza de date în timp și simultan
  - O aplicație poate folosi date ca și cum ar fi singura care le utilizează, fără a ști că altă aplicație, concurent, le poate modifica *(concept de SGBD, veți intra în detalii anul viitor)*
- **Securitatea și confidențialitatea datelor**
  - Datele trebuie protejate de un acces neautorizat sau rău intenționat (prin mecanisme de identificare și **autentificare** a utilizatorilor)
  - Sistemul de gestiune trebuie să asigure securitatea fizică și logică a informației și să garanteze că **numai utilizatorii autorizați** pot efectua operații corecte asupra bazei de date

1.5.2 Dezavantajele SGBD-urilor

Un sistem de gestiune a bazelor de date are o **serie de limitări / dezavantaje** inerente:

- **complexitatea şi dimensiunea sistemelor** pot să crească considerabil, din cauza necesităţii extinderii funcţionalităţilor sistemului
- **costul**, care variază în funcţie de mediu şi funcţionalitatea oferită, la care se adaugă cheltuieli periodice de întreţinere
- **costuri adiționale** pentru elemente de *hardware*
- **costul conversiei** aplicaţiilor existente, necesară pentru ca acestea să poată funcţiona în noua configuraţie *hardware* şi *software*
- **impactul unei defecţiuni** asupra aplicaţiilor, bazei de date sau sistemului de gestiune

1.5.3 Structura unui SGBD pe module

- **programe de gestiune a bazei de date** (**PGBD**) care realizează accesul fizic la date ca urmare a unei comenzi
- **module pentru tratarea LDD** ce permit traducerea unor informaţii în obiecte ce pot fi apoi exploatate în manieră procedurală sau neprocedurală
- **module pentru tratarea LMD** care permit utilizatorilor inserarea, ştergerea, reactualizarea sau consultarea informaţiei dintr-o bază de date
- **module utilitare** care asigură întreținerea, prelucrarea, exploatarea corectă și ușoară a bazei de date
- **module de control** care permit controlul programelor de aplicaţie, asigurarea confidenţialităţii şi integrităţii datelor, rezolvarea unor probleme de concurenţă, recuperarea informaţiei în cazul unor avarii sau defecţiuni *hardware* sau *software*

1.5.4 Arhitectura SGBD

Necesitatea independenței fizice și logice a datelor a determinat adoptarea unei arhitecturi de baze de date organizată pe trei niveluri:

- **nivelul intern** (baza de date fizică, structura internă de stocare a datelor) schema internă permite descrierea datelor unei baze sub forma în care sunt stocate în memoria calculatorului;
- **nivelul conceptual (central)** structura canonică a datelor ce caracterizează procesul de modelat (modelul conceptual, schema conceptuală, structura semantică a datelor fără implementarea pe calculator); **schema conceptuală** permite:
  - definirea tipurilor de date ce caracterizează proprietățile elementare ale entităților
  - definirea tipurilor de date compuse care permit regruparea atributelor pentru a descrie entitățile modelului și legăturile între aceste entități
  - definirea regulilor pe care trebuie să le respecte datele
- **nivelul extern** (modelul extern, subschema, vizualizarea):
  - **Schema externă** reprezintă o descriere a unei părți a bazei de date ce corespunde viziunii unui program sau unui utilizator
  - Modelul extern folosit este dependent de limbajul utilizat pentru prelucrarea bazei de date
  - Permite **asigurarea unei securități** a datelor: un grup de lucru va accesa doar datele descrise în schema sa externă, iar restul datelor sunt protejate

Deci, pe scurt, la nivel **conceptual sau intern**, schemele descriu o bază de date, pe când la **nivel extern** schemele descriu doar o parte din date care prezintă interes pentru un utilizator sau un grup de utilizatori. Pentru o bază de date particulară există o singură schemă internă, o singură schemă conceptuală, dar există mai multe scheme externe.

1.6 Evoluția Bazelor de Date: De la Ierarhic la Suport Decizional

Sistemele de gestiune au evoluat în trei mari generații:

1. **Sisteme ierarhice și rețea:** Reprezentau datele ca arbori sau grafuri. Aveau independență fizică slabă, programatorul fiind obligat să cunoască și să codeze explicit drumurile de acces.
2. **Sisteme relaționale (1970):** Tratează entitățile ca tabele (relații). Nu folosesc pointeri vizibili, au structuri intuitive și se bazează pe operatori matematici. Totuși, au limitări în gestionarea obiectelor complexe sau dinamice.
3. **Sisteme avansate:**
   - **Baze de date orientate obiect (SGBDOO) și relațional-obiect (SGBDROO):** Combină structurile de date complexe, ierarhia de clase și moștenirea cu limbajul relațional.
   - **Baze de date deductive:** Gestionează date (fapte) și reguli de deducție (asertiuni) care permit deducerea de noi fapte prin programare logică.
   - **Baze de date distribuite:** Administrează și prelucrează date situate pe calculatoare diferite, interconectate în rețea. Utilizatorul le vede ca pe o singură bază de date.

OLTP vs. Data Warehouse (OLAP)

O evoluție critică este diferențierea dintre bazele de date tranzacționale (operaționale) și cele cu suport decizional (analitice).

Sistemele OLTP	Data Warehouse (Magazie de Date)
Păstrează date curente, dinamice	Păstrează date istorice, în mare măsură statice
Stochează date detaliate	Stochează date agregate
Prelucrare repetitivă, condusă prin tranzacții cu tipar previzibil	Prelucrare ad-hoc (OLAP), condusă prin analiză cu tipar imprevizibil
Susțin deciziile de zi cu zi, deservesc mulți utilizatori	Susțin deciziile strategice, deservesc puțini utilizatori (management)

Pentru descoperirea tendințelor în aceste mari magazii de date se folosește procesul de **Data Mining** sau Knowledge Discovery in Databases (KDD).

Arhitectura cu mai multe niveluri *(multitier)* a sistemului *Oracle* conține următoarele elemente:

- unul sau mai mulți *clienți* care inițiază operații
- unul sau mai multe *servere* de aplicații care execută părți ale operațiilor
- un *server* de baze de date care stochează datele folosite de operații
- privilegiile *server*-ului de aplicații sunt limitate pentru a preveni execuția operațiilor nedorite sau inutile în timpul unei operații *client*

2 Cursurile 2, 3: Proiectarea Bazelor de Date (Modelarea E/R)

Proiectarea unei baze de date reprezintă trecerea de la percepția unor fapte din lumea reală la reprezentarea lor prin date structurate. Această proiectare se împarte logic (independent de hardware/software) și fizic.

2.1 Modelarea Entitate-Relație

2.1.1 Modelul de Date: Definiție și Componente

Un **model de date** este o colecție integrată de concepte necesare descrierii datelor, a relațiilor dintre ele și a constrângerilor existente în sistemul real analizat. În esență, **modelarea unei baze de date** reprezintă trecerea de la **percepția** unor fapte din lumea reală la reprezentarea acestora prin **date**. Orice model complet trebuie să **reflecte** fidel fenomene ale lumii reale, să **urmărească evoluția** acestei lumi și să permită **comunicarea** dintre fenomenele lumii reale; acesta are 3 componente fundamentale:

1. O parte structurală: reguli conform cărora se construiesc datele.
2. O parte de prelucrare: operații permise pentru regăsirea și actualizarea datelor.
3. Reguli de integritate: constrângeri care asigură coerența logică.

2.1.2 Modelarea semantică a datelor

1. Se identifică o **mulțime de concepte semantice** care sunt utile în descrierea lumii reale. Se presupune că lumea reală *(modelul real analizat)* este formată din **entități** care au anumite **proprietăți**, că fiecare entitate are o **identitate**, că există **legături**, corelații între entități. Conceptul de corelație, ca și cel de entitate, este util, în mod intuitiv, la descrierea modelului.
2. Se caută o **mulțime de obiecte formale**, simbolice, care sunt utiliate pentru reprezentarea conceptelor semantice anterioare.
3. Se dau **reguli de integritate formale și generale** (constrângeri) care să reflecte restricțiile la care este supus modelul.
4. Se definește o **mulțime de operatori formali** prin care pot fi prelucrate și analizate obiectele formale

2.2 Diagramele Entitate-Relație (E/R)

Introdus de Peter Chen în 1976, modelul Entitate-Relație este un model neformalizat, intuitiv, care împarte elementele unui sistem în **Entități** și **Relații** (asocieri), ambele putând avea **Atribute**. Diagrama E/R este modelul **neformalizat** pentru reprezentarea unui sistem din lumea reală.

Elementele Diagramei E/R

- **Entitățile:** Obiecte sau concepte semnificative (persoană, loc, eveniment). Se reprezintă grafic prin **dreptunghiuri** și sunt denumite prin *substantive* (ex: SALARIAT, PROIECT). În modelul relațional, devin tabele.
- **Cheia Primară (PK):** Atribut(e) care identifică unic fiecare apariție a unei entități. Trebuie să fie unică, stabilă și fără ambiguități. Pe diagramă se marchează cu simbolul `#`.
- **Relațiile (Asocierile):** Comunicarea dintre două sau mai multe entități. Sunt exprimate prin *verbe* (ex: SALARIAT *lucrează în* DEPARTAMENT) și sunt reprezentate prin linii care unesc entitățile.
- **Cardinalitatea:** Stabilește numărul minim și maxim de asocieri permise. Cardinalitatea minimă (în paranteză) arată dacă participarea e obligatorie (1) sau opțională (0). Cardinalitatea maximă arată dacă un element se asociază cu unul (1) sau cu mai multe (M / N).
- **Atributele:** Proprietăți descriptive (substantive) aparținând entităților sau chiar relațiilor; trebuie făcută distincția între **atribut** (devine coloană în modelele relaționale) și **valoarea acestuia** (devine valoare în coloane)

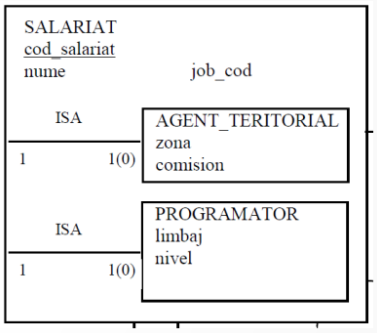
2.3 Cazuri Speciale în Modelarea E/R

În lumea reală, nu toate obiectele sunt simple. Există situații particulare de modelare:

1. Entitatea Dependentă (Slabă): O entitate a cărei existență depinde direct de o altă entitate. De exemplu, entitatea SARCINA nu poate exista fără un PROIECT. În acest caz, **Cheia Primară a entității dependente va include obligatoriu Cheia Primară a entității părinte** + un atribut propriu. Grafic, se desenează folosind linii distincte (mai subțiri sau duble).

2. Relații Recursive: O relație între o entitate și ea însăși. Exemplul clasic este relația de *management*: Un ANGAJAT *este managerul* altui ANGAJAT. Implementarea presupune ca tabelul să aibă o Cheie Externă care arată spre propria sa Cheie Primară (ex: `MANAGER_ID` referă `EMPLOYEE_ID`).

3. Moștenirea atributelor (sub-entitate / super-entitate):



- Subentitate *(subclasă)* - submulțime a unei alte entități, numită superentitate *(superclasă)*
- Subentitatea se desenează prin dreptunghiurile incluse în superentitate
- Există o relație între o subentitate și o superentitate, numită **ISA**, care are cardinalitatea maximă 1 : 1 (fiecare entitate din subclasă **corespunde exact unei** entități din superclasă) și

minimă 1 : 0 (o entitate din superclasă poate **să nu aibă o corespondență** în subclasă)

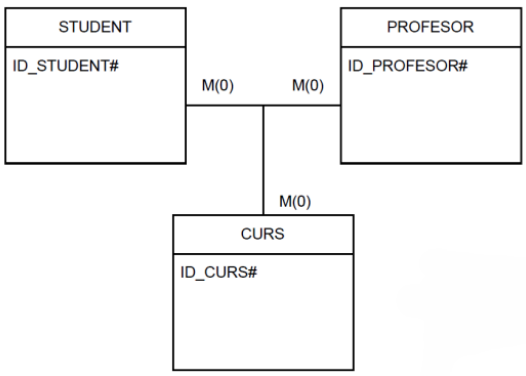
- **Nu orice entitate din superclasă trebuie să aibă o reprezentare într-o subclasă**, dar fiecare entitate dintr-o subclasă **trebuie să fie legată** de o entitate din superclasă.
- De exemplu, într-o relație $SALARIAT \rightarrow PROGRAMATOR$, **nu toți salariații sunt programatori**, dar orice programator **este** și un salariat.
- **Superclasa (superentitatea)** este o entitate care include **subclase (subentități)** distincte, ce trebuie reprezentate în modelul de date.
- **Subclasa** are un rol distinct și, evident, este membră a unei superclase. O subclasă, fiind o entitate, poate să posede propriile subclase.
- O entitate împreună cu subclasele ei, subclasele acestora ș.a.m.d. definește o **ierarhie de tip (ierarhie de specializare)**.

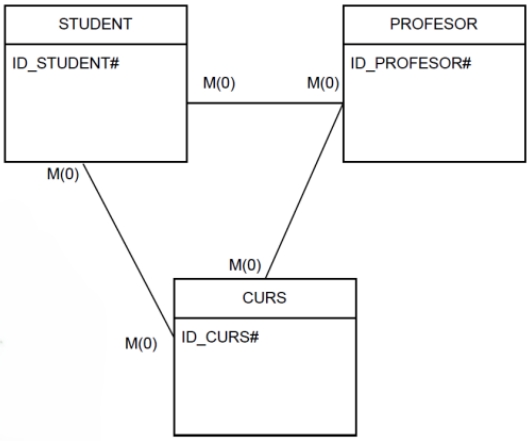
- Cheile primare, atributele și relațiile unei superentități sunt valabile pentru orice subentitate. Afirmația reciprocă este falsă

4. Generalizare, specializare:
Specializarea - de la general la specific

- Este procesul de **maximizare a diferențelor** dintre membrii unei entități, prin identificarea caracteristicilor distinctive ale acestora *(subclasa)*.
- Specializarea reprezintă **procesul de împărțire** a unei superclase în **subclase mai specifice**. Cu alte cuvinte, se pornește de la un concept **general** și se **detaliază** în categorii mai restrânse *(e.g.: entitatea $SALARIAT$ (superclasă) este împărțită în subclase mai specifice: $PROGRAMATOR$, HR, $CONTABIL$).*
- Se aplică atunci când o entitate are subtipuri specifice care trebuie modelate distinct.

5. Relații de tip 2, 3: Unele relații sunt relative la două entități și le numim de **tip 2**, iar dacă relațiile implică **mai mult de două entități**, le vom numi de **tip 3**. Bineînțeles, trei relații de tip 2 sunt diferite de o relație de tip 3; rupând o relație de tip 3 în trei relații de tip 2, pot apărea informații incorecte.





În acest caz, avem **many-to-many** între toate entitățile:

- **Un student poate participa la mai multe cursuri**
- **Un profesor poate preda mai multe cursuri**
- **Un curs poate fi predat de mai mulți profesori**
- Acest lucru confirmă că avem o **relație de tip 3 (ternară)** între *STUDENT*, *CURS* **și** *PROFESOR*

Cardinalitatea minimă: Dacă relația este many-to-many între toate entitățile, atunci cardinalitatea minimă trebuie să fie 0 pentru fiecare entitate, din următoarele motive:

- Un student poate să nu fie înscris la niciun curs (ex: student nou admis care încă nu și-aa ales cursurile).
- Un profesor poate să nu predea niciun curs într-un semestru (ex: concediu, cercetare).
- Un curs poate să nu aibă niciun student înscris (ex: curs nou introdus, dar fără înscrieri).

Implementarea corectă într-o bază de date

Având în vedere că avem **relații many-to-many între toate entitățile**, trebuie să introducem o **tabelă intermediară** pentru relația de tip 3. Această abordare evită dublările de date, păstrează corect relația între cele 3 entități și permite implementarea corectă:

```
CREATE TABLE INSCRIERE (
    ID_STUDENT  NUMBER(10),
    ID_CURS     NUMBER(10),
    ID_PROFESOR NUMBER(10),
    PRIMARY KEY (ID_STUDENT, ID_CURS, ID_PROFESOR),
    FOREIGN KEY (ID_STUDENT) REFERENCES STUDENT(ID_STUDENT),
    FOREIGN KEY (ID_CURS) REFERENCES CURS(ID_CURS),
    FOREIGN KEY (ID_PROFESOR) REFERENCES PROFESOR(ID_PROFESOR)
);
```

În momentul în care se încearcă transformarea relației de tip 3 în 3 relații de tip 2, rezultă o ambiguitate la nivelul relației dintre *STUDENT* și *PROFESOR*, deoarece nu se va mai cunoaște cu exactitate **ce profesor predă un anumit curs unui anumit student**.

6. Excluderea relațiilor indirecte: trebuie **excluse** din model **relațiile indirecte** (ex: un angajat este coleg de departament cu un alt angajat) deoarece ele pot conduce la redundanță în baza de date.

7. Eliminarea atributelor derivabile: Atributele derivabile trebuie eliminate și introduse expresii prin care aceste atribute pot fi calculate:

- **Vârsta** → se calculează din data nașterii

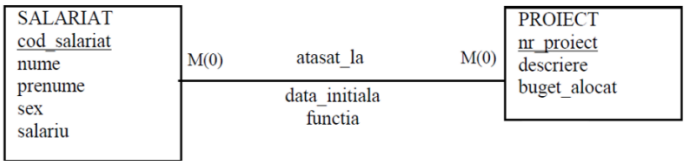
- **Media generală** → se calculează din notele obținute la examene
- **Număr de studenți înscriși la un curs** → se calculează prin numărarea studenților asociați cursului
- **Vechime** → se calculează din data angajării

Aceste atribute derivabile pot fi recalculate oricând *(nu se va ocupa spațiul de stocare cu informații redundante)*, se reduce riscul de inconsistență *(dacă se schimbă datele de bază, atunci vârsta se va recalcula corect)*, iar performanța este optimizată *(se evită actualizările manuale asupra unor date care pot fi generate dinamic)*.

Totuși, dacă un atribut *derivabil* **este folosit foarte des** și **calcularea lui ar încetini performanța** sau dacă **nu se dorește recalcularea datelor istorice**, iar un atribut poate varia în timp, atunci acesta poate fi stocat.

8. Relație sau atribut?: Dacă un atribut al unei entități reprezintă cheia primară a unei alte entități, atunci el referă o relație *(ex: cod_departament în tabelul SALARIAT)*.

9. Entitate sau relație?: Se cercetează cheia primară; dacă aceasta combină cheile primare a două entități, atunci este vorba de o relație *(ex: cheia primară a relației atasat_la combină cod_salariat cu nr_proiect, prin urmare, SALARIAT_atasat_la_PROIECT va defini o relație și nu o entitate)*.



10. Reasignarea atributelor directe:
Un atribut indirect nu descrie real relația sau entitatea, fiind un caz special de relație indirectă care trebuie eliminată. Prin urmare, atributele indirecte trebuie reasignate deoarece introduc redundanță în date *(ex: numărul clădirii în care lucrează un salariat este un atribut al entității DEPARTAMENT și nu este o caracteristică a entității SALARIAT)*.

11. Atribute opționale: Există **atribute opționale**, a căror valoare este **uneori necunoscută** (ex: nu avem informația la momentul respectiv), **alteori neaplicabilă** (ex: nu se aplică pentru toate entitățile); aceste atribute trebuie introduse la subentități.

În cadrul exemplului avem o superentitate *ANGAJAT*, cu două subentități - *PROGRAMATOR* și *MANAGER*:

- Un **atribut opțional** poate fi → **Tehnologie** / **Limbaj**: aplicabil pentru **Programatori**, **neaplicabil** pentru **Manageri**
- Un alt **atribut opțional** poate fi → **Buget gestionat**: aplicabil pentru **Manageri**, **neaplicabil** pentru **Programatori**

2.4 Algoritmul de Proiectare E/R - Exemplu Practic

Pentru a construi corect o diagramă, se parcurg 4 etape clare: 1. Identificarea **entităților** din cadrul sistemului analizat; 2. Identificarea **relațiilor** și stabilirea **cardinalităților**; 3. Identificarea **atributelor** aferente entităților și asocierilor dintre entități; 4. Stabilirea atributelor de identificare a entităților, adică stabilirea **cheilor primare**.

Analiza unui Campionat de Fotbal

Reguli de funcționare: O ECHIPĂ poate fi sponsorizată de mai mulți SPONSORI, suma depinzând de asociația echipă-sponsor. Un CAMPIONAT are mai multe ETAPE, unde se joacă mai multe MECIURI. O ECHIPĂ joacă mai multe MECIURI, iar un MECI e jucat de două echipe.

Aplicarea Algoritmului:

- **Entități:** ECHIPA, SPONSOR, MECI, ETAPA, CAMPIONAT.
- **Relații & Atribute asociative:** SPONSOR *susține* ECHIPA. Deoarece un sponsor susține mai multe echipe și o echipă are mai mulți sponsori (relație M:N), relația va stoca atributul „suma". ECHIPA *joacă* MECI.
- **Entități dependente (Dependențe lanț):**
  - O ETAPA depinde direct de un CAMPIONAT. Cheia primară pentru ETAPA va fi **compusă** (`cod_camp#` + `cod_etapa#`).
  - Un MECI depinde direct de ETAPA în care se joacă. Deci cheia primară a meciului va moșteni cheile părintelui și va deveni: `cod_camp#` + `cod_etapa#` + `cod_meci#`.

2.5 Modelul E/R Extins

Aceeași realitate poate fi percepută diferit de către diverși analiști pentru același sistem, implicând **modele structurale distincte**. Construirea **diagramei conceptuale**, obținerea **schemelor relaționale** și **normalizarea** acestora rezultă într-un **model relațional** care va elimina anumite clase de anomalii ce pot apărea în proiectarea modelului de date.

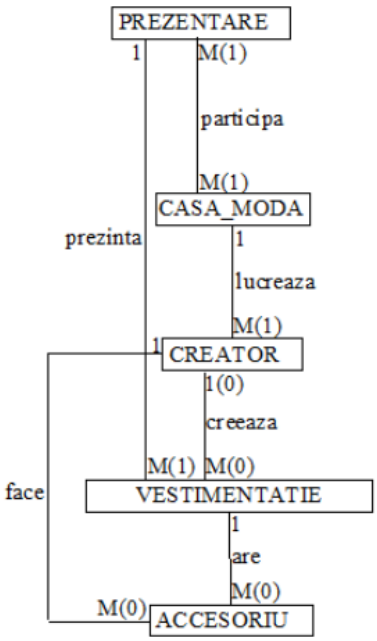
- Conceptele de bază ale modelării E/R nu sunt suficiente pentru a reprezenta cerințe complexe
- Modelul E/R susținut cu concepte semantice adiționale definește **modelul E/R extins (EER)**.
  - include toate **conceptele modelului original**
  - adaugă conceptele de **subclasă, superclasă, moștenire, specializare, generalizare**

2.6 Deficiențe ale modelului E/R

- Interpretare eronată = **capcane de conectare (de întrerupere sau în evantai)**
- Posibil să necesite restructurarea modelului

Capcana de întrerupere

- Modelul sugerează existența unei relații între entități, dar nu există o cale între anumite apariții ale entităților
- Această capcană poate apărea acolo unde există o relație cu participare parțială (0 la card. min.) care face parte din calea dintre entitățile ce sunt legate
- Astfel, modelul de date **nu poate să răspundă la anumite interogări**.



Capcana în evantai

- Modelul ia în considerare o relație între entități, dar calea dintre anumite apariții ale entităților este ambiguă
- Această capcană apare când **două sau mai multe relații one-to-many provin din aceeași entitate**

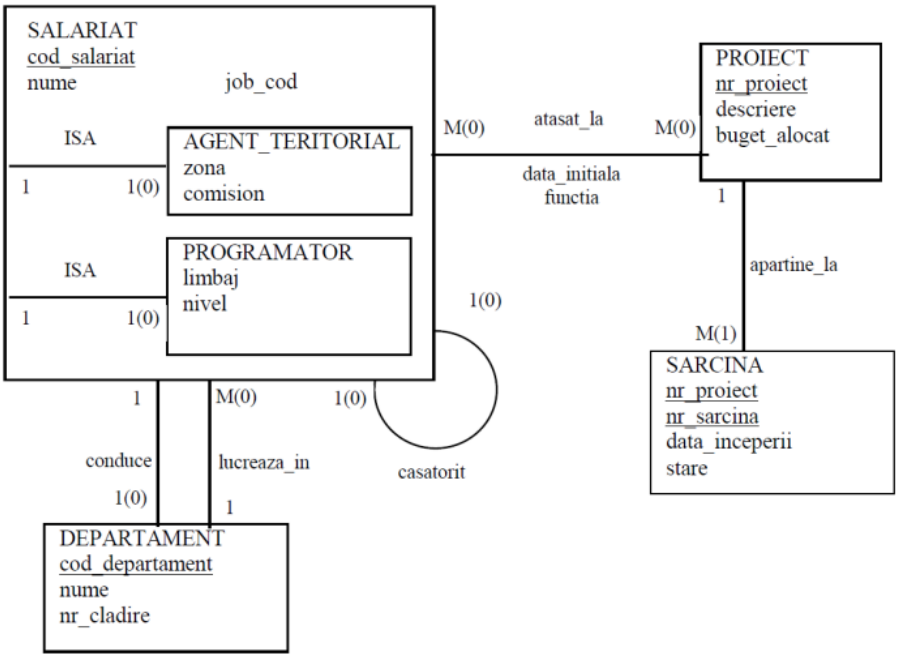


Figura 1: Diagrama E/R a angajaților unei companii

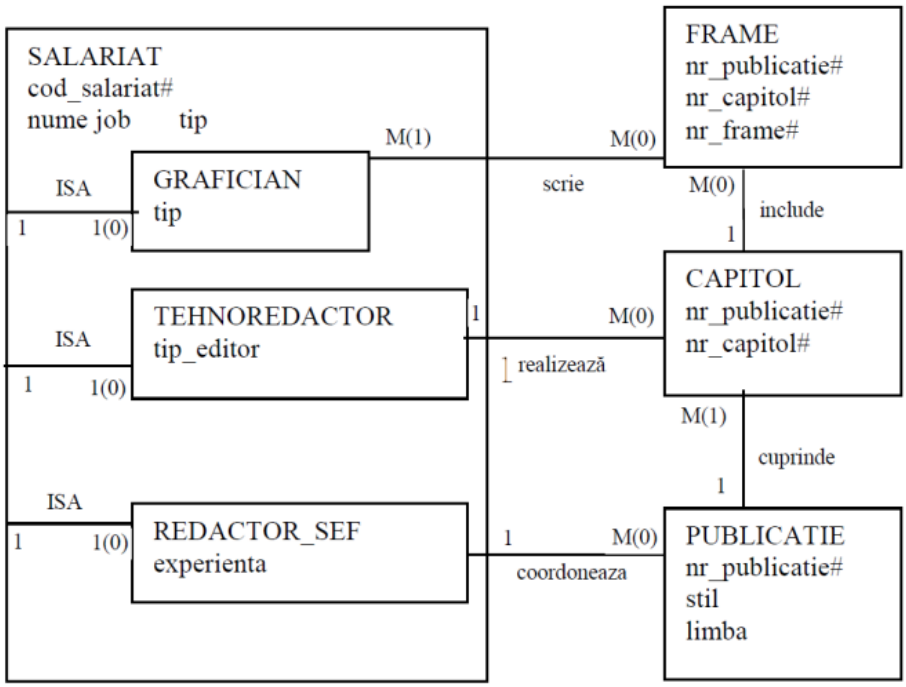


Figura 2: Diagrama E/R a unei edituri

3 Curs 4

3.1 Modelul relațional

MODEL FORMAL de organizare conceptuală a datelor, destinat reprezentării legăturilor dintre date, bazat pe **teoria matematică a relațiilor**; estealcătuit numai din relații (prin urmare, orice interogare asupra BD este o relație)

- **structura** relațională a datelor;
- **operatorii** modelului relațional (specifici algebrei relaționale);
- **regulile de integritate** care guvernează folosirea cheilor în model (constrângeri interne);

FORMAL	UZUAL	FIZIC
Relație	Tablou / Tabel	Fișier
Tuplu	Linie	Înregistrare
Atribut	Coloană	Câmp
Domeniu	Tip de date	Tip de date

- **DOMENIU** – mulţime de valori care poate fi definită fie enumerând elementele componente, fie definind o proprietate distinctivă a domeniului valorilor.
- Fie $D_1, D_2, \ldots, D_n$ domenii finite, nu neapărat disjuncte. **Produsul cartezian** $D_1 \times D_2 \times \cdots \times D_n$ este definit de mulțimea tuplurilor $(V_1, V_2, \ldots, V_n)$ unde $V_i \in D_i$ pentru $i = 1, 2, \ldots, n$, iar **n** definește aritatea tuplului.
- O **relație** pe mulțimile $D_1, D_2, \ldots, D_n$ este o submulțime a produsului cartezian $D_1 \times D_2 \times \cdots \times D_n$; putem reprezenta o relație printr-un **tabel bidimensional**, în care fiecare

coloană corespunde de fapt unui **atribut**.

- Numărul atributelor definește **aritatea** relației, iar numărul de tupluri definește **cardinalitatea** acesteia.
- **Domeniu** = mulțime de val. care pot fi date unui atribut.
- **Relație** = tabel
- **Numele relației** = denumirea tabelului
- **Tuplu** = succesiune de valori de diferite tipuri (un rând al tabelului)
- **Atribut** = coloană a tabelului

3.1.1 Constrângeri de integritate

- aserțiuni pe care datele conținute într-o relație trebuie să le satisfacă
- **reguli structurale** - impun restricții asupra structurii relației
- **reguli de funcționare** - logica aplicației care utilizează baza de date

Există **trei tipuri de constrângeri** structurale (constituie **mulțimea minimală de reguli de integritate** pe care **trebuie** să le respecte un SGBD relațional):

1. **unicitatea cheii:** cheia primară trebuie să fie **unică** și **minimală**
2. **integritatea entității:** atributele cheii primare trebuie să fie **diferite de NULL**
3. **integritatea referirii:** o cheie externă trebuie să fie ori **NULL în întregime**, ori să corespundă unei **valori a cheii primare asociate**.

Restricțiile de integritate minimale sunt definite în raport cu noțiunea de cheie a unei relații.

- Fiecare relație are cel puțin o cheie;
- Una dintre **cheile candidat** va fi aleasă drept **cheie primară**;
- Cheia primară **NU poate fi reactualizată**;
- O **cheie** identifică linii **ȘI ESTE DIFERITĂ de un index**; indexul este folosit pentru a localiza rândurile rapid, fără o parcurgere în întregime a tabelului.

1. **Cheia primară**(Primary Key) este folosită pentru a identifica UNIC fiecare rând(tuplu) într-un tabel;
2. **O cheie secundară**(Secondary Key) este folosită ca index pentru a accesa tupluri - aceasta nu identifică unic fiecare rând, dar poate fi folosită ca INDEX;
3. **Cheia externă**(Foreign Key) este **un atribut care creează o relație** între două tabele, referind **PK** a altui tabel;
4. **O supercheie** este un grup de atribute **PK**, reprezentând cheia primară a tabelului ce descrie **o relație**

3.2 Diagrama conceptuală

3.2.1 Transformarea entităților

- Ent. independente devin **tabele independente** (cheia primară nu conține **chei externe**)
- Ent. dependente devin **tabele dependente** (cheia primară a acestora conține **PK** a ent. de care depinde + unul sau mai multe atribute adiționale)

- Subent. devin **subtabele** (cheia externă referă **cheia primară din supertabel**)

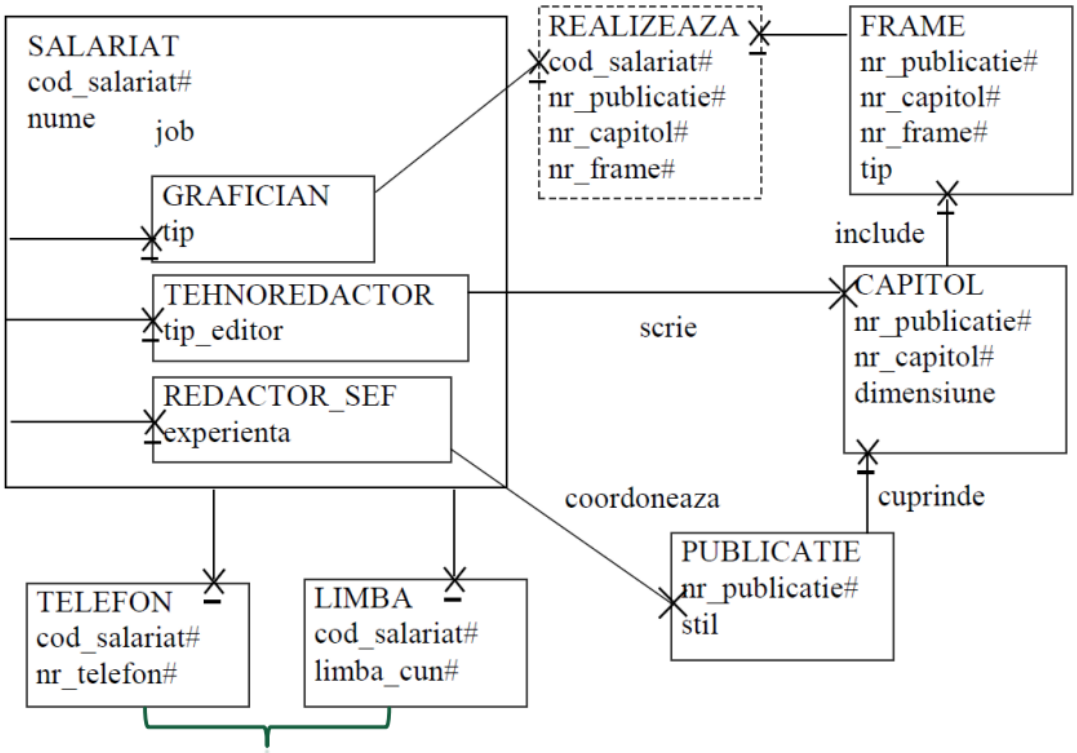
3.2.2 Transformarea relațiilor

- Relațiile **1:1 și 1:N** devin **chei externe** (simbolul "×" indică plasamentul cheii externe, iar simbolul "$\underline{\times}$" exprimă faptul că această cheie externă este conținută în cheia primară); relația 1:1 plasează **FK** în tabelul cu mai puține linii;
- Relațiile **M:N** devin tabele speciale, numite **tabele asociative**, care au două chei externe pentru cele două tabele asociate (cheia primară a tabelului este compunerea acestor două chei externe + eventuale coloane adiționale);
- Relațiile de tip trei devin **tabele asociative** (cheia primară este compunerea a trei chei externe + eventuale coloane adiționale).

3.2.3 Transformarea atributelor

- Un atribut singular devine o **coloană**;
- Atributele multiple devin **tabele dependente** ce conțin cheia primară a entității și atributul multiplu (cheia primară este o cheie externă + una sau mai multe coloane adiționale);
- Pentru **relații 1:1 și 1:n**, atributele relațiilor vor aparține tabelului care conține cheia externă; pentru **relații m:n și de tipul trei** atributele vor fi plasate în tabelele asociative.

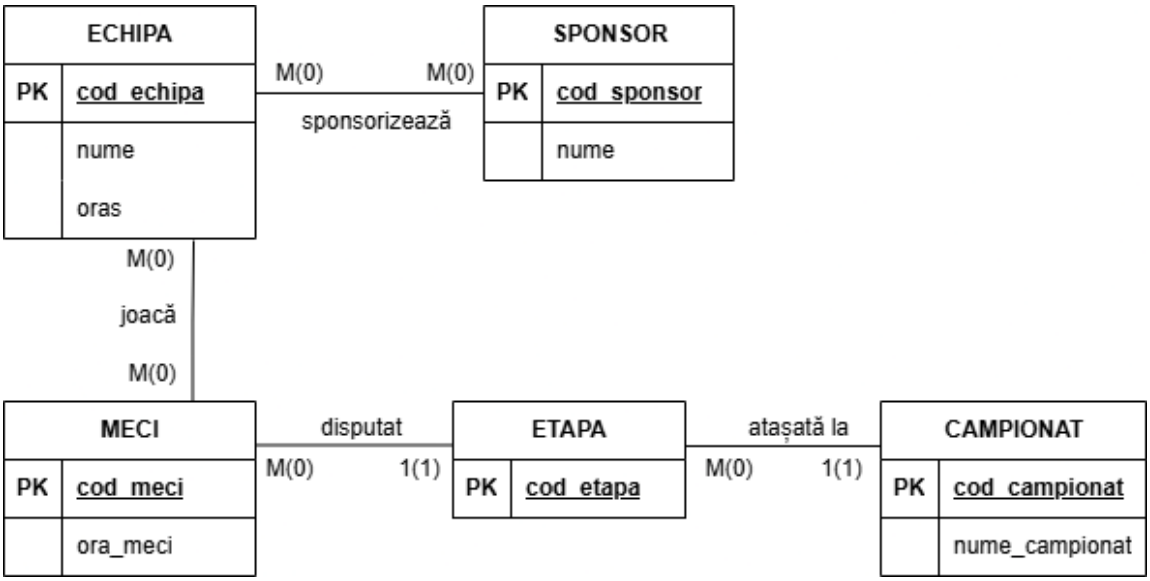
Tabel	Reprezintă	Cheie primară
Independent	entitate independentă	nu conține FK
Subtabel	Subentitate	o cheie externă
Dependent	entit. dependentă atribute multiple	o cheie externă și una sau mai multe coloane adiționale
Asociativ	relație m:n relații de tip 3	două sau mai multe chei externe și (opțional) adiționale coloane

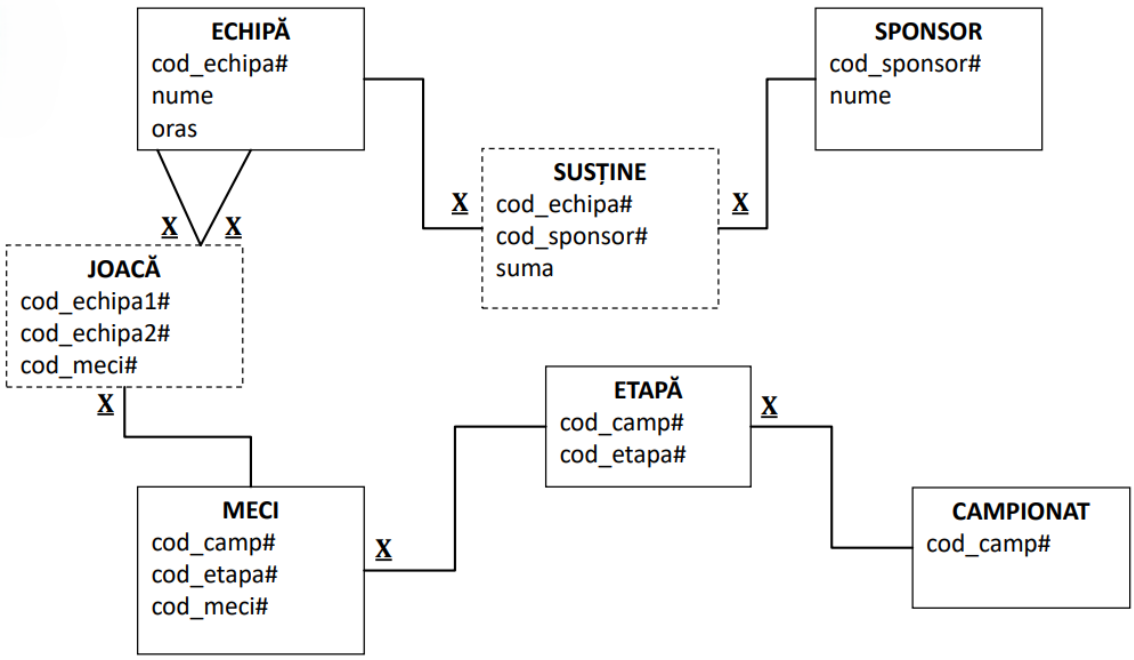


Cum se procedează într-un astfel de caz?

Pentru a nu avea redundanță în cadrul tabelului $LIMBA$, o soluție satisfăcătoare ar fi să fie creat un tabel separat pentru limbile vorbite, $LIMBI_VORBITE$, în care fiecare limbă vorbită să fie adăugată o singură dată, urmând ca acest tabel să intre într-o relație de tip 2 cu tabelul salariat; astfel, în diagrama conceptuală, acest tabel asociativ $SALARIAT_LIMBI_VORBITE$ ar permite o cheie primară compusă; în cazul tabelului $TELEFON$, acesta necesită o descriere mai amănunțită a situațiilor pe care acesta ar trebui să le trateze: dacă unui salariat îi pot fi asociate **mai multe** numere de telefon, atunci este validă această implementare, fiind stabilită o relație de **one-to-many** *(1:M)* între $SALARIAT$ și $TELEFON$, iar în caz contrar, acest tabel poate să fie redus la nivelul unui **atribut opțional**.

4 Curs 6: Exemplu de Diagramă Conceptuală





Sfaturi Design BD

- Introducem o **cheie primară artificială** *(altă denumire este cea de **surrogate key**)* dacă avem o cheie primară compusă din multe atribute (în practică dacă sunt mai mult de 3-4 atribute).
- Introducerea unei **chei primare** artificiale conduce la pierderea dependenței.
- Se poate pune cheie artificială și pe tabelul asociativ, caz în care tabelul rămâne punctat în Diagrama Conceptuală, însă fără dependențe **('X' simplu, nu subliniat)**.

- În cazul atributelor multiple alegem modelarea ca tabel dependent sau relație $M : M$? Dacă aceeași valoare poate aparține mai multor entități, atunci poate fi o opțiune de design relația $M : M$.

5 Curs 7: Algebra Relațională

LDD - precizează **entitățile**, relațiile dintre ele, **atributele**, structura atributelor, **cheile**, **constrângerile**, prin urmare definește **structura obiectelor bazei de date** (**schema** bazei de date)

LMD - cuprinde aspecte referitoare la introducerea, modificarea, eliminarea și căutarea datelor;

Modelul relaţional oferă **două mulţimi de operatori pe relaţii**:

- **algebra relaţională** metodă **procedurală** - descrie **cum** se obține rezultatul. Se folosesc **operatori** pentru a manipula relațiile (filtrele se obţin aplicând operatori specializaţi asupra uneia sau mai multor relaţii din cadrul bazei relaţionale);
- **calculul relaţional** metodă care descrie **ce** condiții trebuie să îndeplinească rezultatul, fără să specifice cum se obține (filtrele se obţin cu ajutorul unor formule logice pe care tuplurile rezultatului trebuie să le satisfacă);

Echivalenţa dintre algebra relaţională şi calculul relaţional a fost demonstrată de Jeffrey David Ullman. Această echivalenţă arată că orice relaţie posibil de definit în algebra relaţională poate fi definită şi în cadrul calcului relaţional, şi reciproc.

STUDENȚI

id#	nume	id_facultate
1	Andrei	10
2	Maria	20
3	Ioana	10

FACULTĂȚI

id_facultate#	nume_facultate
10	Informatica
20	Litere

1. Algebra relațională:

$R1 = JOIN(STUDENTI, FACULTATI,$
$STUDENTI.id_facultate = FACULTATI.id_facultate)$

$R2 = SELECT(R1, nume_facultate =' Informatica')$

$Rezultat = PROJECT(R2, nume);$

2. Calculul relațional:

```
{ s.nume |
  s IN STUDENTI AND
  f IN FACULTATI AND
  s.id_facultate = f.id_facultate AND
  f.nume_facultate = 'Informatica'
}
```

- **Algebra relațională** are la bază o **mulțime de operații** care se aplică asupra unor relații, rezultatul fiind tot o relație;
- Pentru a scrie cereri folosind **algebra relațională** avem nevoie de simboluri specifice, operații și implicit operatori;
- Prin algebra relațională, se pot construi cererile pas cu pas până la obținerea unui rezultat final, prin scrierea **expresiilor relaționale**

Operatorii algebrei relaționale:

- operatori clasici pe mulțimi **(UNION, INTERSECT, PRODUCT, DIFFERENCE)**
- operatori relaționali speciali **(PROJECT, SELECT, JOIN, DIVISION)**

5.1 Operatorul PROJECT

- Se utilizează atunci când se dorește preluarea anumitor valori din baza de date, fără a fi necesară o condiție.
- Operație unară care elimină anumite atribute ale unei relații, producând o submulțime "pe verticală" a acesteia.

Notații:

5.2 Operatorul SELECT

- Se utilizează atunci când preluăm date pe baza unei condiții
- Selecția (restricția) este o operație unară care produce o submulțime "pe orizontală" a unei relații R

Exemplu: Să se obțină toate informațiile despre angajații care au jobul 'IT_PROG'.

Cerere SQL:

```
SELECT * FROM employees WHERE job_id = 'IT_PROG';
```

Algebra relațională (expresia algebrică):
Rezultat = SELECT(EMPLOYEES, job_id = 'IT_PROG')

5.3 Operatorul UNION

- Reuniunea a două relații R și S (elementele comune și necomune afișate o singură dată)
- **Reuniunea** a două relații este mulțimea tuplurilor aparținând fie lui R, fie lui S, fie ambelor relații.

Notații:

- **UNION(R,S)**
- $R \cup S$
- OR(R,S)
- APPEND(R,S)

Exemplu: Se cer codurile departamentelor al căror nume conține șirul 're' sau în care lucrează angajați având codul job-ului 'SA_REP'.

Cerere SQL:

```
SELECT department_id FROM departments WHERE LOWER(department_name) like
    '%re%'
UNION
```

```
SELECT department_id FROM employees WHERE UPPER(job_id) like '%SA_REP%'
    ;
```

Algebra relațională (expresia algebrică):
R1 = SELECT(DEPARTMENTS, department_id LIKE '%re%')
R2 = PROJECT(R1, department_id)
R3 = SELECT(EMPLOYEES, job_id LIKE '%SA_REP%')
R4 = PROJECT(R3, department_id)
Rezultat = UNION(R2,R4)

5.4 Operatorul DIFFERENCE

- diferența a două relații R și S (elementele care sunt în R și nu sunt în S) - mulțimea tuplurilor care aparțin lui R, dar nu și lui S.
- Diferența este o operație binară necomutativă care permite obținerea tuplurilor ce apar numai într-o relație.

Notații:

- **DIFFERENCE(R,S)**
- R - S
- REMOVE(R,S)
- MINUS(R,S)

Exemplu: Să se obțină codurile departamentelor în care nu lucrează nimeni.

Cerere SQL:

```
    SELECT department_id FROM departments MINUS SELECT department_id
   FROM employees;
```

Algebra relațională (expresia algebrică):
R1 = PROJECT(DEPARTMENTS, department_id)
R2 = PROJECT(EMPLOYEES, department_id)
Rezultat = DIFFERENCE(R1,R2)

5.5 Operatorul INTERSECT

- Intersecția a două relații R și S (elementele comune, afișate o singură dată)
- Mulțimea tuplurilor care aparțin atât lui R, cât și lui S

Notații:

- **INTERSECT(R,S)**
- $R \cap S$

- AND(R,S)

Exemplu: Să se afileze angajații (codul lor) care sunt manageri în cadrul departamentelor și în același timp sunt și managerii altori angajați.

Cerere SQL:

```
SELECT manager_id FROM departments
INTERSECT
SELECT manager_id FROM employees;
```

Algebra relațională (expresia algebrică):
R1 = PROJECT(DEPARTMENTS, manager_id)
R2 = PROJECT(EMPLOYEES, manager_id)
Rezultat = INTERSECT(R1,R2)

5.6 Operatorul PRODUCT

- Produsul cartezian dintre relația R și relația S
- Fie R și S de aritate **m**, respectiv **n**. Produsul cartezian al lui R cu S este mulțimea tuplurilor de aritate **m** + **n**, unde primele m componente formează un tuplu în R, iar ultimele n componente formează un tuplu în S.

Notații:

- **PRODUCT(R,S)**
- $R \times S$
- TIMES(R,S)

Exemplu: Să se afișeze toți angajații împreună cu toate departamentele.

Cerere SQL:

```
SELECT employee_id, last_name, d.department_id, department_name
FROM employees e, departments d;
```

Algebra relațională (expresia algebrică):
R1 = PROJECT(EMPLOYEES, employee_id, last_name)
R2 = PROJECT(DEPARTMENTS, department_id, department_name)
Rezultat = PRODUCT(R1,R2)

5.7 Operatorul JOIN

Permite regăsirea informației din mai multe relații corelate. Operatorul combină **produsul cartezian** cu **selecția** și **proiecția**. 4 tipuri de **JOIN**:

- **NATURAL JOIN**

- θ-**JOIN**
- **SEMI-JOIN**
- **OUTER JOIN**

5.7.1 NATURAL JOIN

Operatorul de compunere naturală combină tupluri din două relații R și S, cu condiția ca **atributele comune să aibă valori identice** (coloanele comune sunt cele care au același nume în ambele tabele). **Algoritmul care realizează compunerea naturală este următorul:**

1. se calculează produsul cartezian $\mathbf{R} \times \mathbf{S}$;
2. pentru fiecare atribut comun **A** care definește o coloană în R și o coloană în S, se selectează din $R \times S$ tuplurile ale căror valori coincid în coloanele **R.A** și **S.A**;
3. pentru fiecare astfel de atribut **A** se elimină coloana S.A, iar coloana R.A se va numi A.

Operatorul poate fi exprimat formal astfel:

$$\mathbf{JOIN(R,\ S)} = \mathbf{\Pi}_{i1,\ldots,im}\ \boldsymbol{\sigma}_{(R.A1=S.A1)\wedge\cdots\wedge(R.Ak=S.Ak)}(\mathbf{R} \times \mathbf{S})$$

Exemplu: Să se afișeze informații despre angajații care lucrează în departamente și care sunt în același timp atât manageri de departament, cât și managerii altor angajați.

Cerere SQL:

```
SELECT employee_id, last_name, d.department_id, department_name, d.
   manager_id
FROM employees e, departments d
WHERE e.department_id = d.department_id AND
      e.manager_id = d.manager_id;
```

Algebra relațională (expresia algebrică):
Rezultat = JOIN(EMPLOYEES, DEPARTAMENTS)

5.7.2 θ-JOIN

Combină tupluri(coloane / atribute) din două relații R și S pe baza unei condiții specificate în cadrul operației de JOIN. Această condiție poate fi bazată pe egalitatea unor col. (de obicei **PK** și **FK**: cazul **EQUIJOIN**, iar în cazul unei inegalități (prin alți operatori), **NONEQUIJOIN**).

$$\mathbf{JOIN(R,\ S,\ condiție)} = \boldsymbol{\sigma}_{\mathbf{condiție}}\ (\mathbf{R} \times \mathbf{S})$$

Exemplu: Să se afișeze numele angajaților împreună cu denumirea jobului în cadrul căruia lucrează.

Cerere SQL:

```
SELECT last_name, job_title
FROM employees e, jobs j
WHERE e.job_id = j.job_id;
```

Algebra relațională (expresia algebrică):
R1 = PROJECT(EMPLOYEES, last_name, job_id)
R2 = PROJECT(JOBS, job_title, job_id)
R3 = JOIN(R1, R2, EMPLOYEES.job_id = JOBS.job_id)
Rezultat = PROJECT(R3, last_name, job_title)

5.7.3 SEMI-JOIN

- Se utilizează atunci când din cele două relații participante R și S se utilizează doar atributele relației R sau doar cele ale relației S.
- Operatorul SEMI-JOIN conservă **atributele unei singure relații participante** la compunere și este utilizat când nu sunt necesare toate atributele compunerii; operator derivat, fiind o combinație de proiecție și compunere (naturală sau theta);

Notație:

- **SEMIJOIN(R,S) = PROJECT(JOIN(R,S))**
- **SEMIJOIN(R,S, cond) = PROJECT(JOIN(R,S,cond))**

Exemplu: Să se afișeze departamentele care au cel puțin un angajat.

Cerere SQL:

```
SELECT d.department_id, department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id;
```

Algebra relațională (expresia algebrică):
R1 = JOIN(EMPLOYEES, DEPARTAMENTS, EMPLOYEES.department_id = DEPARTMENTS.department_id)
Rezultat = PROJECT(R1, department_id, departament_name)

5.7.4 OUTER JOIN

LEFT / RIGHT / FULL (OUTER) JOIN **Exemplu:** Să se afișeze angajații împreună cu departamentele în care lucrează, inclusiv și angajații care nu aparțin unui departament.

5.8 Operatorul DIVISION

Diviziunea este o operație binară care definește o relație ce conține valorile atributelor dintr-o relație care apar în toate valorile atributelor din cealaltă relație.

Notații:

- **DIVISION(R,S)**
- DIVIDE(R,S)

Diviziunea conține acele tupluri de dimensiune n - m la care, adăgând orice tuplu din S, se obține un tuplu din R.

- $\mathbf{R}^{(n)} \div \mathbf{S}^{(m)} = \{\mathbf{t}^{(n-m)} \mid \forall s \in \mathbf{S}, (t, s) \in \mathbf{R}\}$, unde $n > m$ și $S \neq \emptyset$.

Cuantificatorul universal nu există în SQL, dar poate fi însă simulat cu ajutorul dublei negații și a cuantificatorului existențial:

$$\forall x\, \mathbf{P}(x) \equiv \neg \exists x\, \neg \mathbf{P}(x).$$

6 Curs 8: Evaluarea și Optimizarea Interogărilor

6.1 Procesarea interogărilor

- O **expresie a algebrei relaționale** este constituită din **relații** legate prin **operații** din algebra relațională;
- O expresie se poate reprezenta **grafic** cu ajutorul unui **arbore**, numit **arbore algebric**, în care nodurile corespund operatorilor din cadrul expresiei respective;
- Evaluarea unei expresii presupune efectuarea prelucrărilor indicate de operatorii din expresie în ordinea apariţiilor sau în ordinea fixată prin paranteze;
- **Rezultatul** evaluării unei expresii este o **relație derivată** din relațiile menționate ca operanzi în cadrul expresiei;
- Două expresii sunt echivalente, dacă în urma evaluării lor se obține ca rezultat aceeași relație

În majoritatea sistemelor de gestiune, în special în cele relaționale, interfața cu utilizatorul este de tip **neprocedural**; utilizatorul definește datele pe care dorește să le vizualizeze fără a da algoritmii de acces (**CE**, nu **cum**). Sistemul trebuie să convertească cererea utilizatorului într-o cerere optimală, respectiv în proceduri de acces optimal la datele fizice. Garantarea absolută a performanțelor optime pentru procesorul limbajului relațional este totuși imposibilă.

Orice cerere SQL este prelucrată în trei etape fundamentale:

1. **Analiza cererii (Parsing):** Verificare sintactică (dacă e corect scrisă) și semantică (dacă tabelele/coloanele există și userul are drepturi).
2. **Ordonarea și Optimizarea (Optimization):** SGBD-ul construiește un "arbore algebric", apoi alege planul de execuție cu **cel mai mic cost estimativ**.
3. **Execuția (Execution):** Administratorul de date caută blocurile fizice pe disc, le aduce în memoria cache și returnează rezultatul.

Operațiile sunt, în general, cele ale **algebrei relaționale** (SELECT, PROJECT, JOIN, DIVISION, UNION, INTERSECT, PROJECT, DIFFERENCE), după care se stabilește o ordine de execuție optimă a acestor operații; la sfârșitul etapei se obține un plan de execuție al cererii. **Execuția** presupune **efectuarea** (paralelă și/sau secvențială a) operațiilor elementare furnizate de planul de execuție pentru a obține **rezultatul cererii**.

Presupunem că utilizatorul transmite sistemului de gestiune o **cerere** exprimată prin **comenzi SQL**; pentru a răspunde cererii, SGBD-ul trebuie să *înțeleagă* cererea utilizatorului, aceasta necesitând: să fie corectă sintactic, să aibă **datele disponibile** utilizatorului și să localizeze datele analizând diferite **drumuri de acces** la ele.

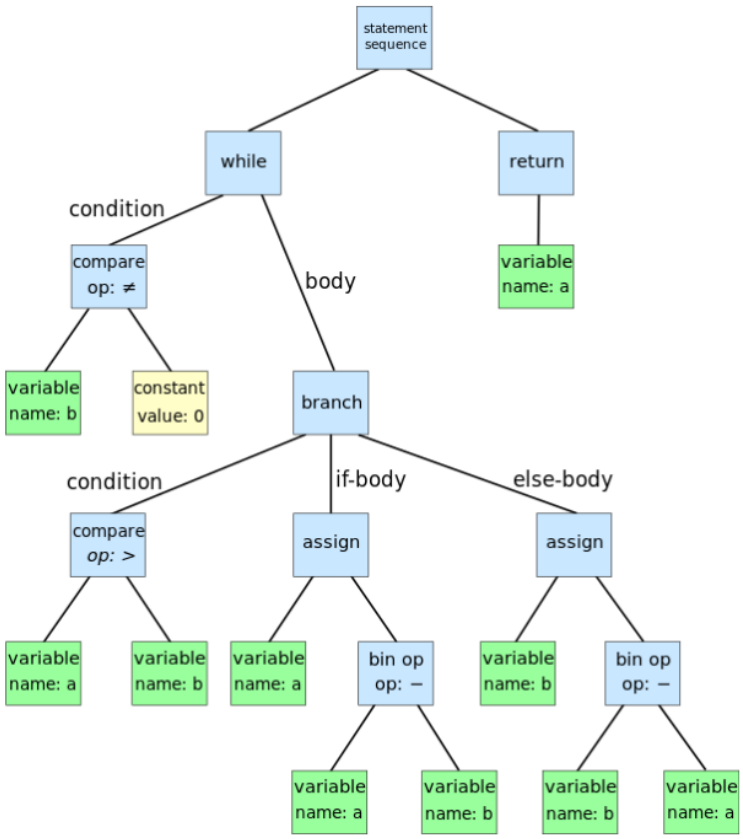
- **Analizorul cererilor:** Asigură verificarea *sintactică și semantică* a cererii. De asemenea, localizează datele implicate pe disc și furnizează *planul de execuție*, decizând cea mai eficientă metodă de a extrage datele.

- **Administratorul datelor (executorul):** Primește și execută efectiv planurile de execuție furnizate de modulul de optimizare. Această execuție presupune căutarea blocurilor de date și transferul lor în *memoria cache*.

- Datele **nu** sunt citite direct de pe disc la fiecare interogare, procesul fiind unul lent.
- Acestea sunt copiate în **memoria cache** (buffer pool), o zonă rapidă din RAM gestionată de SGBD, care acționează ca un intermediar.
- Dacă aceeași cerere (sau una similară) este executată din nou, datele sunt returnate instantaneu din cache, ceea ce *crește semnificativ viteza de răspuns*.

- Pentru procesarea interogărilor, compilatorul efectuează trei tipuri de analiză: *lexicală, sintactică și semantică*.
- Rolul său este să verifice dacă programul compilează și rulează fără erori. **Erorile semantice** reprezintă erori de compilare care nu sunt de sintaxă (ex: tipuri de date incompatibile sau erori de conversie/acces).
- În final, pentru realizarea tuturor acestor verificări, compilatorul generează un **arbore**.

```
while b ≠ 0
  if a > b
    a = a - b
  else
    b = b - a
return a
```



Compilatorul SQL execută o cere într-un mod similar, convertind cererea utilizatorului într-o variantă optimizată, cu ajutorul unui arbore algebric. **Idee generală: cerere → arbore algebric (nu este unic) → plan de execuție → optimizare**

- Cu ajutorul **planului de execuție**, se realizează o evaluare a cererii în vederea obținerii unor *optimizări*.
- Deoarece **arborele algebric** nu este unic, pot exista *planuri de execuție diferite* care să producă în final același rezultat.
- Scopul optimizării este acela de a identifica **planul de execuție optim**, adică planul de *cost minim*.
- Un **plan de execuție** implică o **secvență de pași** pentru evaluarea cererii (în mod obișnuit, fiecare pas corespunde unei *operații relaționale*).
- Totodată, implică alegerea **metodei** ce va fi folosită pentru evaluarea operației, dat fiind că pot exista mai multe metode pentru a evalua o aceeași operație relațională.

- Două planuri de execuție diferite care conduc întotdeauna la același rezultat poartă numele de **planuri echivalente**.
- Aceste *planuri de execuție echivalente* pot avea **costuri diferite**.
- Obiectivul procesului de optimizare a cererilor este de a selecta, dintre diversele planuri echivalente, pe cel cu *cost minim*.
- Într-un sistem centralizat, **costul evaluării** unei cereri este reprezentat de suma a două componente:
  - **costul I/O** (care reprezintă transferurile de date);
  - **costul CPU** (care implică verificarea de condiții, operațiile de *join* etc.).

6.2 Ordinea de execuție a operațiilor

- O interogare este formată dintr-un număr de operații; ordinea în care se efectuează operațiile are un rol important în evaluarea costului necesar realizării interogării;
- Există **două modalități** de abordare pentru a determina ordinea de execuție a operațiilor: algebric și bazat pe estimarea costului;
- Ambele folosesc o mulțime de **reguli** care permit **transformarea unui plan de execuție** (reprezentat ca o expresie scrisă în termenii algebrei relaționale) în altul, echivalent;
- Astfel, o **cerere SQL** se poate scrie sub forma unei **expresii a algebrei relaționale**, formată din relații și operații specifice. Ulterior, aceasta se poate reprezenta grafic sub forma unui **arbore algebric**, în care nodurile corespund operatorilor algebrei relaționale utilizați în cadrul cererii, urmând etapa de evaluare și optimizare a acesteia;
- Optimizarea cererilor, utilizând algebra relațională, se realizează pornind de la scrierea cererii sub forma unor **expresii algebrice**, după care se aplică transformări echivalente, dar mai eficiente;
- Pentru optimizare se utilizează și **proprietățile operatorilor algebrei relaționale**.
-

6.3 Proprietățile operatorilor algebrei relaționale. Optimizare

Proprietatea 1. Comutativitatea operațiilor de *join* și produs cartezian

$$JOIN(R1, R2) = JOIN(R2, R1)$$
$$R1 \times R2 = R2 \times R1$$

Proprietatea 2. Asociativitatea operațiilor de *join* și produs cartezian:

$$JOIN(JOIN(R1, R2), R3) = JOIN(R1, JOIN(R2, R3))$$
$$(R1 \times R2) \times R3 = R1 \times (R2 \times R3)$$

Proprietatea 3. Compunerea proiecțiilor:

$$\sqcap_{A_1,\ldots,A_m}(\sqcap_{B_1,\ldots,B_n}(R)) = \sqcap_{A_1,\ldots,A_m}(R)$$

unde $\{A_1, A_2, \ldots, A_m\} \subseteq \{B_1, B_2, \ldots, B_n\}$

Proprietatea 4. Compunerea selecțiilor

$$\sigma_{cond1}(\sigma_{cond2}(R)) = \sigma_{cond1 \wedge cond2}(R) = \sigma_{cond2}(\sigma_{cond1}(R))$$

unde am notat prin *cond* condiția după care se face selecția.

Proprietatea 5. Comutarea selecției cu proiecția:

$$\sqcap_{A_1,\ldots,A_m}(\sigma_{cond}(R)) = \sigma_{cond}(\sqcap_{A_1,\ldots,A_m}(R))$$

unde condiția după care se face selecția implică numai atributele $A_1, \ldots, A_m$.
Dacă condiția implică și atributele $B_1, \ldots, B_n$, care nu aparțin mulțimii $\{A_1, \ldots, A_m\}$, atunci:

$$\sqcap_{A_1,\ldots,A_m}(\sigma_{cond}(R)) = \sqcap_{A_1,\ldots,A_m}(\sigma_{cond}(\sqcap_{A_1,\ldots,A_m,B_1,\ldots,A_n}(R)))$$

Proprietatea 6. Comutarea selecției cu produsul cartezian:

Dacă toate atributele menționate în condiția după care se face selecția sunt atribute ale relației $R1$, atunci:

$$\sigma_{cond}(R1 \times R2) = \sigma_{cond}(R1) \times R2$$

Dacă condiția este de forma $cond1 \wedge cond2$ și dacă $cond1$ implică numai atribute din $R1$, iar $cond2$ implică numai atribute din $R2$, atunci

$$\sigma_{cond}(R1 \times R2) = \sigma_{cond1}(R1) \times \sigma_{cond2}(R2)$$

Dacă $cond1$ implică numai atribute din $R1$, iar $cond2$ implică atribute atât din $R1$ cât și din $R2$, atunci:

$$\sigma_{cond}(R1 \times R2) = \sigma_{cond2}(\sigma_{cond1}(R1) \times R2)$$

Proprietatea 7. Comutarea selecției cu reuniunea:

$$\sigma_{cond}(R1 \cup R2) = \sigma_{cond}(R1) \cup \sigma_{cond}(R2)$$

Proprietatea 8. Comutarea selecției cu diferența:

$$\sigma_{cond}(R1 - R2) = \sigma_{cond}(R1) - \sigma_{cond}(R2)$$

Proprietatea 9. Comutarea proiecției cu produsul cartezian:
Dacă $A_1, ..., A_m$ este o listă de atribute ce apar în schemele relaționale $R1$ și $R2$ și dacă lista este formată din atribute aparținând lui $R1$ (notate prin $B_1, ..., B_n$) și din atribute aparținând lui $R2$ (notate prin $C_1, ..., C_k$ atunci:

$$\sqcap_{A_1,...,A_m}(R1 \times R2) = \sqcap_{B_1,...,B_n} \times \sqcap_{C_1,...,C_k}$$

Proprietatea 10. Computarea proiecției cu reuniunea:

$$\sqcap_{A_1,...,A_m}(R1 \cup R2) = \sqcap_{B_1,...,B_n} \cup \sqcap_{C_1,...,C_k}$$

Proprietatea 11. Compunerea proiecției cu operația *join*:
Dacă $A_1, ..., A_m$ este o listă de atribute ce apar în schemele relaționale $R1$ și $R2$ și dacă lista este formată din atribute aparținând lui $R1$ (notate prin $B_1, ..., B_n$) și din atribute aparținând lui $R2$ (notate prin $C_1, ..., C_k$, atunci:

$$\sqcap_{A_1,...,A_m}(JOIN(R1, R2, D)) = \sqcap_{A_1,...,A_m}(JOIN(R1, R2, D))$$

unde am notat prin $JOIN(R1, R2, D)$ operația de compunere naturală între $R1$ și $R2$ după atributul comun D.

Proprietatea 12. Compunerea selecției cu operația *join*:

$$\sigma_{cond}(JOIN(R1, R2, D)) = \sigma_{cond}(JOIN(\sqcap_{D,A}(R1), \sqcap_{D,A}(R2), D))$$

unde A reprezintă atributele care apar în condiția după care se face selecția

Reguli euristice de optimizare (Transformări Algebrice)

1. **Selecțiile (σ) se execută cât mai devreme posibil:** Reduc substanțial numărul de rânduri transmise mai departe către JOIN-uri costisitoare.

2. **Proiecțiile (π) se execută la început:** Se elimină atributele nefolositoare pentru a reduce memoria consumată.

3. **Evitarea produselor carteziene:** Produsele carteziene ($\times$) se înlocuiesc cu JOIN-uri ori de câte ori este posibil.

4. **Cel mai restrictiv JOIN se execută primul:** Dacă există mai multe tabele, legătura care produce rezultatul cel mai mic se realizează prima.

   - Când sunt mai multe **JOIN-uri** într-o cerere SQL, cel mai restrictiv înseamnă acel JOIN **care filtrează cele mai multe rânduri de la început** – adică lasă în rezultat mai puține date de procesat mai departe.
   - De ce este important? Pentru că atunci când baza de date execută cererea, ea trebuie să combine tabelele. Dacă începe cu un **JOIN care reduce mult volumul de date**, următoarele operații se vor face pe mai puține rânduri, deci vor fi mai rapide și vor consuma mai puține resurse (memorie, CPU etc.).

Exemplu de Arbore Algebric

Cererea SQL:

```
SELECT titlu, valoare
FROM opera o JOIN colectie c ON (o.cod_colectie = c.cod_colectie)
    JOIN restaureaza r ON (o.cod_opera = r.cod_opera)
WHERE upper(o.cod_galerie) = 'G1'
    AND initcap(c.proprietar) - 'King'
    AND r.data > to_date('05/06/2000', 'dd/mm/yyyy');
```

Expresia algebrică: $R1 = SELECT(OPERA, cod_galerie =' G1')$
$R2 = PROJECT(R1, cod_opera, titlu, valoare, cod_colectie)$
$R3 = SELECT(COLECTIE, proprietar =' King')$
$R4 = PROJECT(R3, cod_colectie)$
$R5 = SEMIJOIN(R2, R4, cod_colectie)$
$R6 = SELECT(RESTAUREAZA, data > 05/06/2000)$
$R7 = PROJECT(R6, cod_opera)$
$R8 = SEMIJOIN(R5, R7, cod_opera)$
$Rezultat = R9 = PROJECT(R8, titlu, valoare)$

7 Secțiunea PL/SQL: Variabile și Structuri de Control

PL/SQL (Procedural Language / Structured Query Language) este extensia procedurală Oracle, combinând flexibilitatea SQL-ului cu instrucțiuni decizionale, repetitive și gestionare de erori.

7.1 Tipuri de Date și Declararea Variabilelor

Orice bloc PL/SQL are secțiunea de `DECLARE`. Variabilele pot fi strict tipizate (ex. `NUMBER`, `VARCHAR2`) sau ancorate dinamic la o tabelă.

- **%TYPE:** Preia tipul de date al unei coloane (ex: `v_salariu employees.salary%TYPE;`)
- **%ROWTYPE:** Preia structura unui întreg rând dintr-o tabelă (ex: `v_angajat employees%ROWTYPE;`)

7.2 Structuri Decizionale

Limbajul suportă structuri clasice `IF` și `CASE`.

```
DECLARE
    v_salariu NUMBER := 5000;
BEGIN
    IF v_salariu < 3000 THEN
        DBMS_OUTPUT.PUT_LINE('Salariu mic');
    ELSIF v_salariu BETWEEN 3000 AND 8000 THEN
        DBMS_OUTPUT.PUT_LINE('Salariu mediu');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Salariu mare');
    END IF;
END;
```

7.3 Structuri Repetitive (Bucle)

Există 3 tipuri principale de bucle: **1. LOOP Simplu (Ieșire controlată):**

```
DECLARE
    i NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE('Iteratia: ' || i);
        i := i + 1;
        EXIT WHEN i > 5;
    END LOOP;
END;
```

2. WHILE LOOP:

```
WHILE i <= 5 LOOP
    DBMS_OUTPUT.PUT_LINE('Iteratia: ' || i);
```

```
    i := i + 1;
END LOOP;
```

3. FOR LOOP (Iterație fixă, i este declarat implicit):

```
BEGIN
    FOR i IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE('Iteratia: ' || i);
    END LOOP;
END;
```

7.4 Cursori în PL/SQL

Cursorul este un pointer către o zonă de memorie (Context Area) ce stochează rezultatul unei interogări.

1. Cursori Impliciți: Sunt gestionați automat de Oracle pentru comenzi DML (`INSERT, UPDATE, DELETE`) sau pentru un `SELECT INTO` care returnează exact un single rând. Au atribute precum `SQL%FOUND`, `SQL%NOTFOUND`, `SQL%ROWCOUNT`.

2. Cursori Expliciți: Definiți de programator pentru interogări care returnează **mai multe rânduri**. Pașii sunt: DECLARARE → DESCHIDERE → EXTRAGERE (FETCH) → ÎNCHIDERE.

```
DECLARE
    -- 1. Declararea cursorului
    CURSOR c_angajati IS
        SELECT last_name, salary FROM employees WHERE department_id =
   10;
    v_nume employees.last_name%TYPE;
    v_salariu employees.salary%TYPE;
BEGIN
    -- 2. Deschiderea cursorului
    OPEN c_angajati;
    LOOP
        -- 3. Extragerea rand cu rand
        FETCH c_angajati INTO v_nume, v_salariu;
        EXIT WHEN c_angajati%NOTFOUND; -- Conditia de iesire

        DBMS_OUTPUT.PUT_LINE(v_nume || ' are salariul ' || v_salariu);
    END LOOP;
    -- 4. Inchiderea cursorului
    CLOSE c_angajati;
END;
```

Alternativ, se poate folosi **FOR LOOP pe cursor**, care face automat pașii OPEN, FETCH, CLOSE și declararea variabilei rând:

```
BEGIN
    FOR rec_angajat IN (SELECT last_name FROM employees) LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(rec_angajat.last_name);
    END LOOP;
END;
```

7.5 Gestiunea Excepțiilor

Excepțiile tratează erorile apărute la runtime, prevenind întreruperea bruscă a programului. Secțiunea `EXCEPTION` se plasează la finalul blocului.

- **Predefinite Oracle:** `NO_DATA_FOUND` (SELECT INTO nu a adus niciun rând), `TOO_MANY_ROWS` (SELECT INTO a adus mai multe rânduri), `ZERO_DIVIDE` etc.
- **Definite de utilizator:** Declarate în `DECLARE` și aruncate explicit folosind comanda `RAISE`.

```
DECLARE
    v_nume employees.last_name%TYPE;
BEGIN
    SELECT last_name INTO v_nume FROM employees WHERE employee_id =
   99999;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Eroare: Angajatul nu exista!');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Eroare: Interogarea a returnat mai multi
   angajati!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('A aparut o eroare neasteptata: ' ||
   SQLERRM);
END;
```